

Session 01 – Introduction to AI and Deep Learning

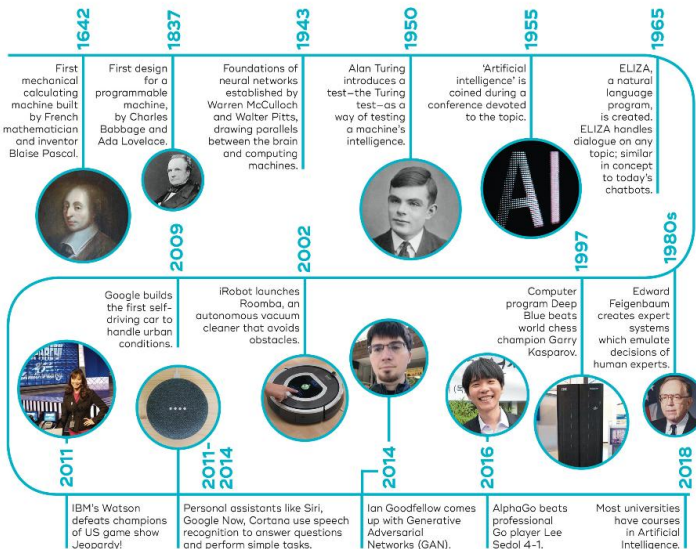
Dr Ivan Olier

I.Olier@ljmu.ac.uk

2026 Summer School in AI

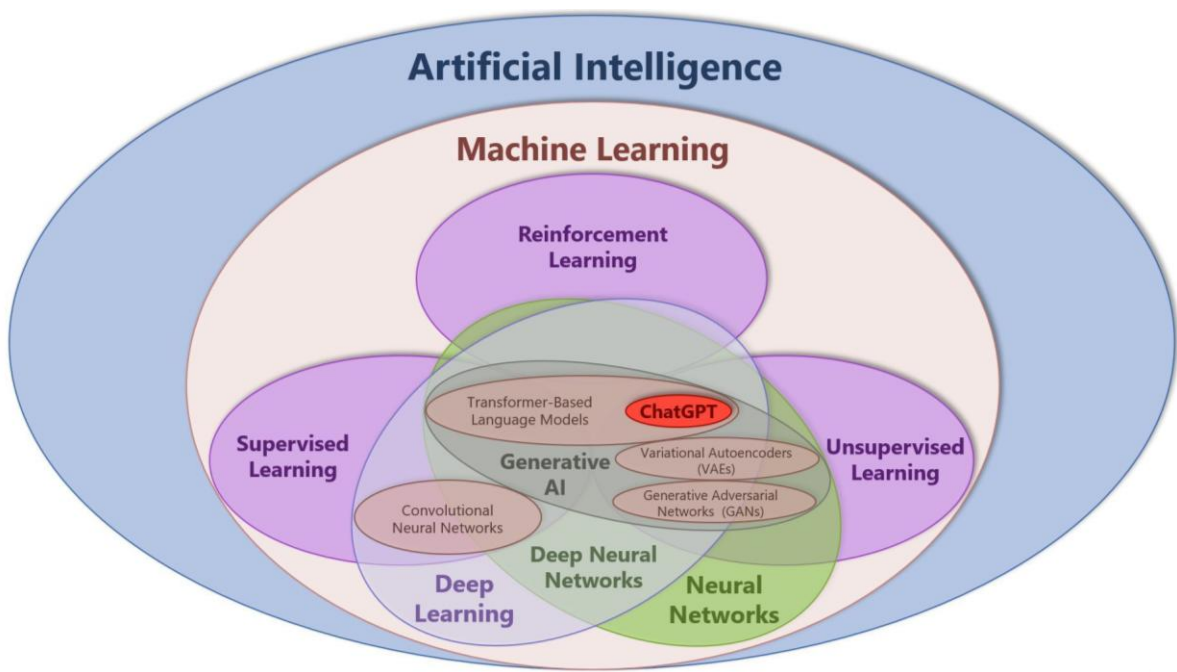
1

Artificial Intelligence



Artificial intelligence (AI) is a collection of technologies that enable machines to perform tasks that typically require human intelligence. AI uses algorithms, data, and computational power to simulate human intelligence, allowing machines to learn, reason, and solve problems.

Google's Gemini

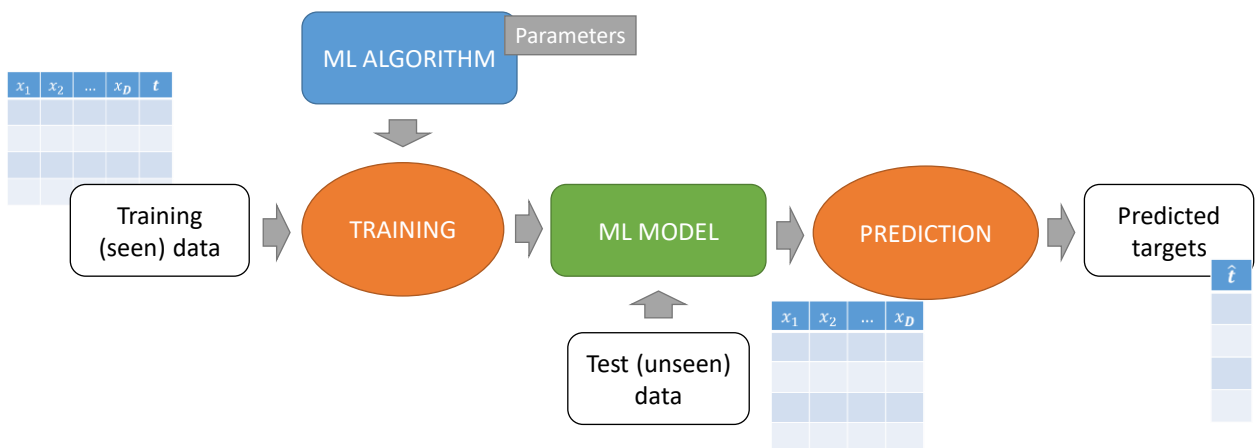


Dr Ivan Olier | AI

3

Model fitting (or training)

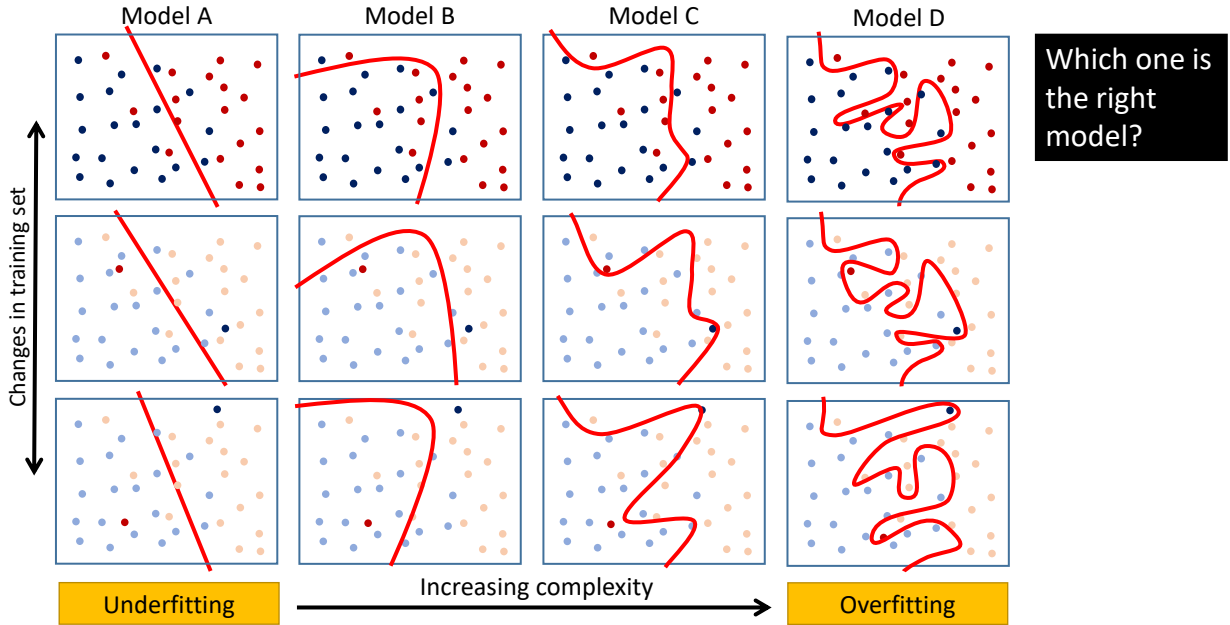
- A ML/Statistical model is **fitted** or trained by using data (training data) to estimate its parameters.
- In turns, a fitted model can be used to make **predictions** using **unseen data** (test data).



Dr Ivan Olier | AI

4

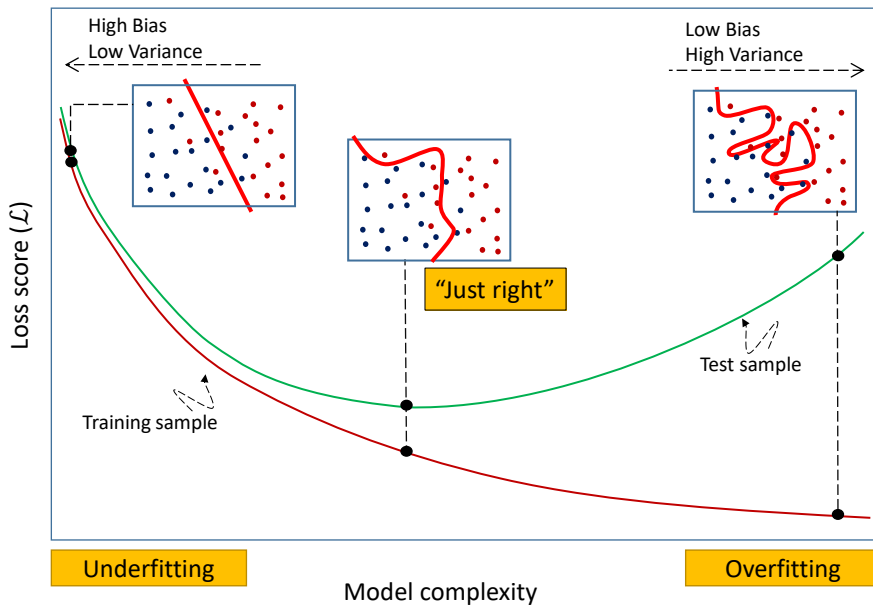
Model selection – classification



Dr Ivan Olier | AI

5

Bias – variance trade-off

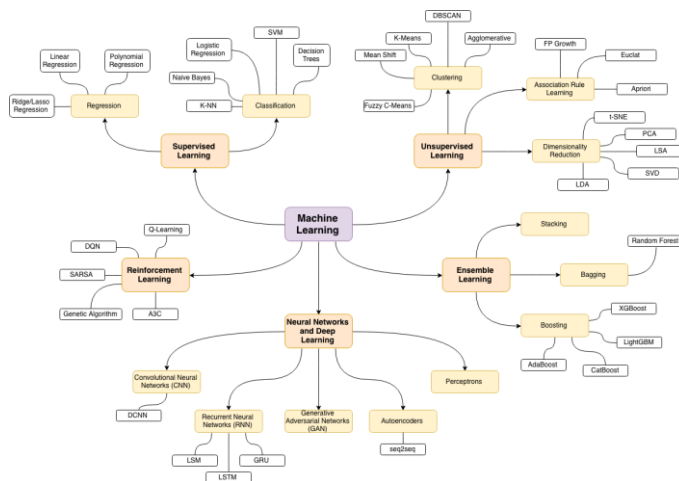


Dr Ivan Olier | AI

6

Machine Learning algorithms

- Logistic regression
- Artificial neural networks (ANNs)
- Support vector machines (SVMs)
- k-Nearest neighbours (kNN)
- Decision trees (CARTs)
- Ensemble learning (bagging, random forest [RF], gradient boosted machines [GBM])



Dr Ivan Olier | AI

7

ANNs learning algorithm

Dr Ivan Olier | AI

8

Definition of (artificial) neural networks

- A neural network is a **massively parallel** distributed processor made up of simple processing units that has a natural propensity for storing experiential knowledge and making it available for use.
- It **resembles** the brain in two respects:
 1. Knowledge is acquired by the network from its environment through a learning process.
 2. Inter-neuron connection strengths, known as synaptic **weights**, are used to store the acquired **knowledge**.

Dr Ivan Olier | AI

9

From linear regression to neurons

Linear regression:

$$y = b + w_1x_1 + w_2x_2 + \dots + w_mx_m$$

$$v = b + \sum_{j=1}^m w_jx_j$$

$$y = v$$

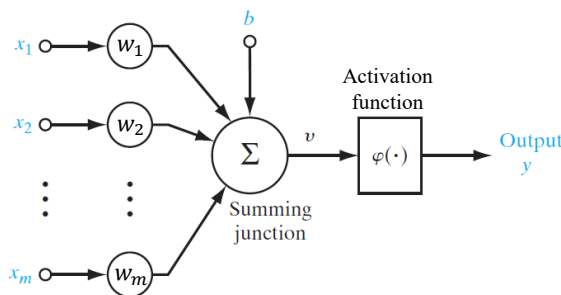
Logistic regression:

$$y = p(x) = \frac{e^{b+w_1x_1+\dots+w_mx_m}}{1 + e^{b+w_1x_1+\dots+w_mx_m}}$$

$$y = p(x) = \frac{1}{1 + e^{-(b+w_1x_1+\dots+w_mx_m)}}$$

$$y = \varphi(v) = \frac{1}{1 + e^{-v}}$$

Graphical representation of the models



This is a **neuron** – a simple processing that performs:

1. Linear combination (adder) of inputs + bias
2. (Non)-linear activation function (φ) to produce the output.

Dr Ivan Olier | AI

10

Summary of commonly used activation functions

Activation function	Equation	Example	1D Graph
Unit step (Heaviside)	$\phi(z) = \begin{cases} 0, & z < 0, \\ 0.5, & z = 0, \\ 1, & z > 0, \end{cases}$	Perceptron variant	
Sign (Signum)	$\phi(z) = \begin{cases} -1, & z < 0, \\ 0, & z = 0, \\ 1, & z > 0, \end{cases}$	Perceptron variant	
Linear	$\phi(z) = z$	Adaline, linear regression	
Piece-wise linear	$\phi(z) = \begin{cases} 1, & z \geq \frac{1}{2}, \\ z + \frac{1}{2}, & -\frac{1}{2} < z < \frac{1}{2}, \\ 0, & z \leq -\frac{1}{2}, \end{cases}$	Support vector machine	
Logistic (sigmoid)	$\phi(z) = \frac{1}{1 + e^{-z}}$	Logistic regression, Multi-layer NN	
Hyperbolic tangent	$\phi(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	Multi-layer Neural Networks	
Rectifier, ReLU (Rectified Linear Unit)	$\phi(z) = \max(0, z)$	Multi-layer Neural Networks	
Rectifier, softplus	$\phi(z) = \ln(1 + e^z)$	Multi-layer Neural Networks	

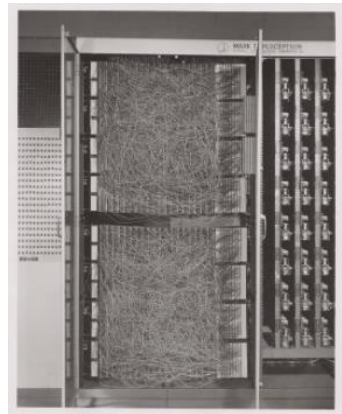
Copyright © Sebastian Raschka 2016
(<http://sebastianraschka.com>)

Dr Ivan Olier | AI

11

The Rosenblatt’s Perceptrons

- **Perceptrons:**
 - the first generation of neural networks.
- They were popularised by Frank Rosenblatt in the early **1960’s**.
 - They appeared to have a very powerful learning algorithm.
 - Lots of grand claims were made for what they could learn to do.
- In 1969, **Minsky and Papert** published a book called “**Perceptrons**” that analysed what they could do and showed their limitations.
 - Many people thought these limitations applied to **all** neural network models.

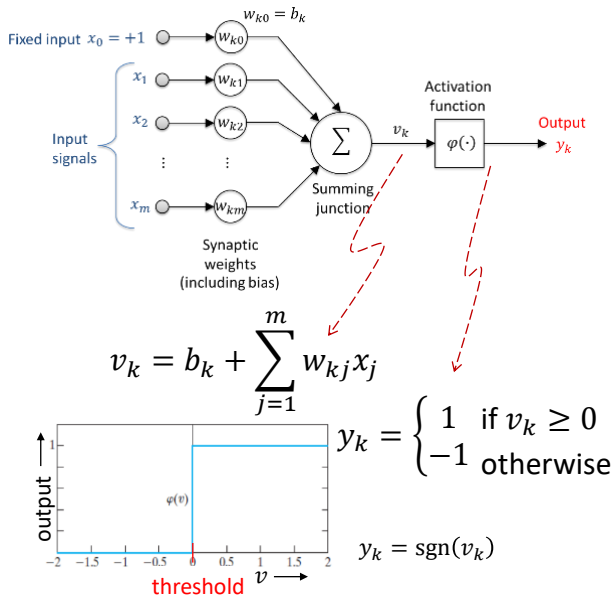


The Mark I Perceptron machine was the 1st implementation of the perceptron algorithm.

Dr Ivan Olier | AI

12

Perceptron as a binary classifier



Steps for training the perceptron as a classifier:

- Pick training cases using any policy that ensures that every training case will keep getting picked.
 - If the output unit is correct, **leave** its weights alone.
 - If the output unit incorrectly outputs a zero, **add** the input vector to the weight vector.
 - If the output unit incorrectly outputs a 1, **subtract** the input vector from the weight vector.

$$\Delta(\mathbf{w}) \propto (t^n - y^n(i)) \mathbf{x}^n$$

Dr Ivan Olier | AI

13

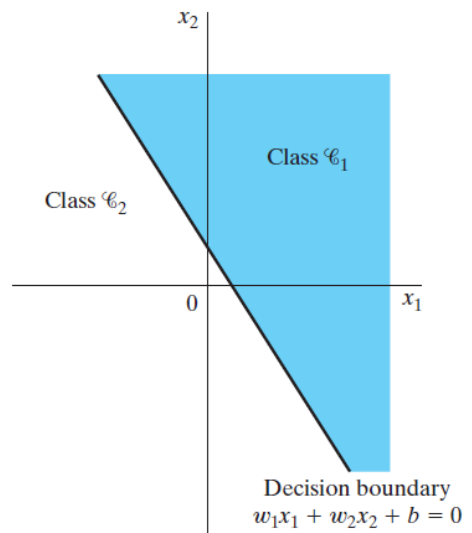
Perceptron as a binary classifier

- The perceptron is a linear classifier.

$$y = \text{sgn } v = \text{sgn} \left[b + \sum_{j=1}^m w_j x_j \right]$$

- The decision boundary is a hyperplane, which splits the data into two classes.

$$0 = b + \sum_{j=1}^m w_j x_j$$

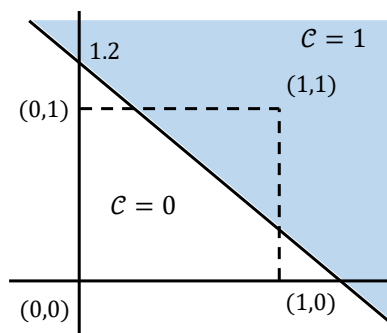


Dr Ivan Olier | AI

14

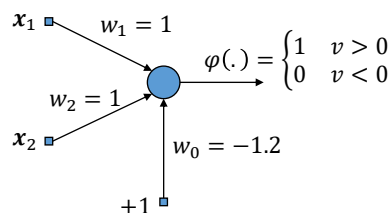
Modelling the “AND” gate with NNs

“AND” gate



Decision boundary:
 $0 = x_1 + x_2 - 1.2$

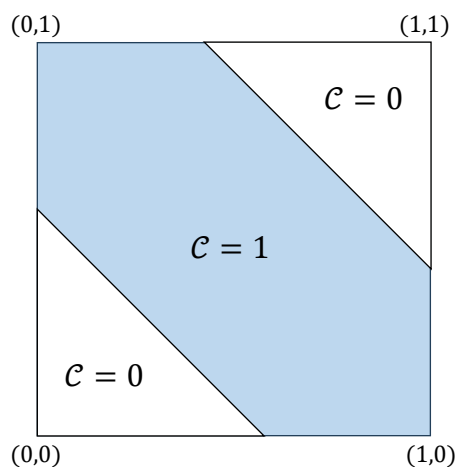
Proposed perceptron



Validation:

x_1	x_2	v	$\mathcal{C} = \varphi(v)$
0	0	-1.2	0
0	1	-0.2	0
1	0	-0.2	0
1	1	0.8	1

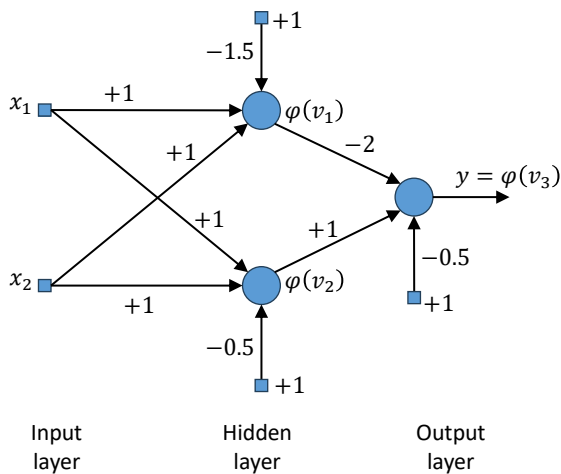
Modelling the “XOR” gate with a Perceptron?



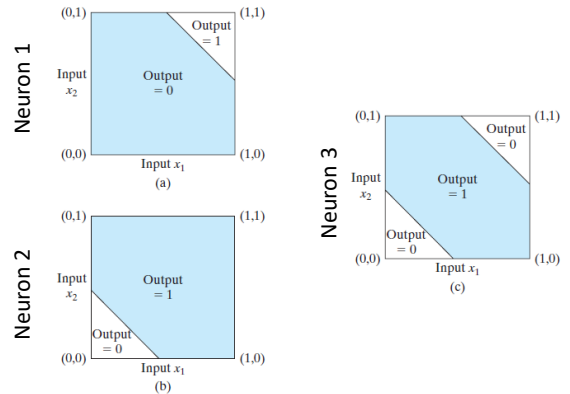
And what about the “XOR” gate?

- It is a non-linear classifier, hence cannot be implemented with a perceptron!

Adding a hidden layer to solve the XOR problem



x_1	x_2	v_1	$\phi(v_1)$	v_2	$\phi(v_2)$	v_3	$o_3 = \phi(v_3)$
0	0	-1.5	0	-0.5	0	-0.5	0
0	1	-0.5	0	0.5	1	0.5	1
1	0	-0.5	0	0.5	1	0.5	1
1	1	0.5	1	1.5	1	-1.5	0



Dr Ivan Olier | AI

17

Multilayer perceptron (MLP)

- Networks without hidden units are very limited in the input-output mappings they can learn to model.
 - More layers of linear units do not help. It is still linear.
 - Fixed output non-linearities are not enough.
- The perceptron convergence procedure works by ensuring that every time the weights change, they get closer to every “generously feasible” set of weights.
 - This type of guarantee cannot be extended to more complex networks in which the average of two good solutions may be a bad solution.
- So “multilayer” neural networks do not use the perceptron learning procedure.
 - They should never have been called multi-layer perceptrons.

We want a method that can be generalised to multi-layer, non-linear neural networks.

Dr Ivan Olier | AI

18

Gradient descent

- If we define the loss (or error) as a function of the weights, $\mathcal{L}(\mathbf{w})$, it makes sense to let the weights update proportionally to loss change:

$$\Delta \mathbf{w}^{(n)} \propto \frac{\partial \mathcal{L}}{\partial \mathbf{w}^{(n)}}$$

where $\Delta \mathbf{w}^{(n)} = \mathbf{w}(n+1) - \mathbf{w}(n)$, and n labels the iteration step, so:

$$\mathbf{w}(n+1) = \mathbf{w}(n) - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{w}^{(n)}}, \quad \text{where } \eta > 0 \text{ is the learning rate.}$$

- After each such update, the gradient is **re-evaluated** for the new weight vector and the process repeated.
- Note that the loss function is defined with respect to a training set, and so each step requires that the entire training set be processed in order to evaluate the gradient.
- The gradient $\frac{\partial \mathcal{L}}{\partial \mathbf{w}^{(n)}}$ is estimated using the **back-propagation** algorithm.

Dr Ivan Olier | AI

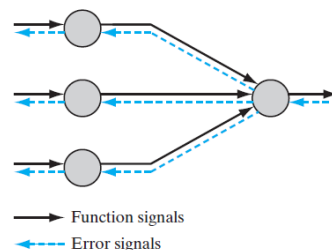
19

The back-propagation algorithm

- We do not know what the hidden units ought to do, but we can compute how fast the error changes as we change a hidden activity on a training case.

- Data pre-processing** – It is recommended to **standardise** the data (mean=0, sd=1).
- 1. Initialisation** – Unless there is prior information available, pick synaptic weights from uniform distribution whose mean is zero and variance smaller than 1 (**assuming** standardised data).
- 2. Presentation** of training examples – Present the network an **epoch** of training examples. For each example in the sample perform the sequence of forward and backward computations:
- 3. Forward computation** – Compute network outputs $\mathbf{y}(n)$ and errors $\mathbf{e}(n)$.
- 4. Backward computation** – Compute the local gradients and then, adjust the weights as follows:

Iteration – Iterate the forward and backward computations under points 3 and 4 by presenting new epochs of training examples to the network until the **chosen stopping criterion** is met (e.g. validation loss convergency).



Dr Ivan Olier | AI

20

An analogy for understanding gradient descent ^[*]

- A person is stuck in the mountains and is trying to get down.



- There is heavy fog and the path down the mountain is **not visible**, so he must use local information to get down.
- He can use the method of **gradient descent**, which involves looking at the **steepness** of the hill at his current position, then proceeding in the direction with the **steepest descent**. Using this method, he would eventually find his way down the mountain.
- Assume that the steepness of the hill is not immediately obvious with simple observation, but rather it requires a **sophisticated instrument to measure**, which the person happens to have at the moment.
- It takes quite some time to measure the steepness of the hill with the instrument, thus he should **minimise** its use. The difficulty then is **choosing the frequency** at which he should measure the steepness of the hill so **not to go off track**.

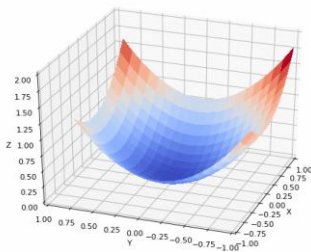
[*] <https://en.wikipedia.org/wiki/Backpropagation>

Dr Ivan Olier | AI

21

An analogy for understanding gradient descent ^[*]

- A person is stuck in the mountains and is trying to get down (i.e. **trying to find the minima**).



- In this analogy:
 - the person: the **backpropagation algorithm**, and the path taken down the mountain represents the sequence of parameter settings that the algorithm will explore.
 - the steepness of the hill: the **slope of the error surface** at that point.
 - the instrument used to measure steepness: **differentiation** (the slope of the error surface can be calculated by taking the derivative of the squared error function at that point).
 - the direction he chooses to travel in: **gradient** of the error surface at that point.
 - the amount of time he travels before taking another measurement: the **learning rate** of the algorithm.

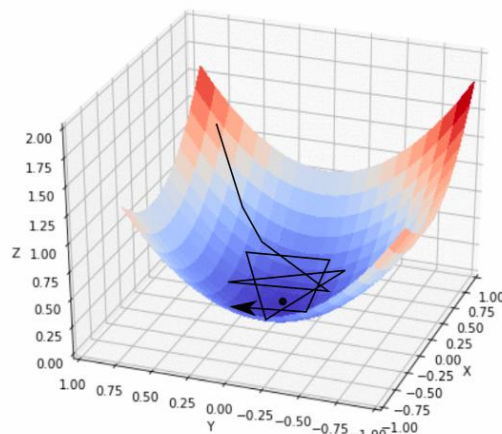
[*] <https://en.wikipedia.org/wiki/Backpropagation>

Dr Ivan Olier | AI

22

Learning rate

- The learning rate **controls** the speed of the learning process.
- Too high: in some cases network could learn fast but better solutions could be missed. Also, the opposite effect could happen: it could keep bouncing along the ridges of the "valley" without ever reaching the minima.
- Too low: the training might turn out to be too long to be feasible at all. Even if that is not the case, very slow learning rates make the algorithm more prone to get stuck in a minima.



Too fast.

Modelling with Neural Networks

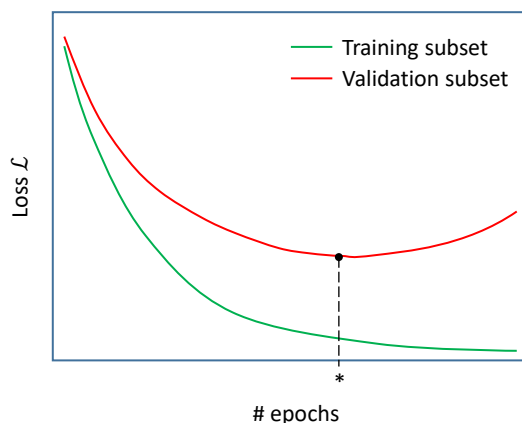
- For now, split your data into training, validation and test subsets.
- Use validation subset to minimise the risk of overfitting.
- Use test subset to report model performance.

- Loss function, $\mathcal{L}(\mathbf{w})$,
 - For continuous targets – any error metric could be used provided that is differentiable. E.g. **Error energy**:

$$\mathcal{L} = E = \frac{1}{2} \sum_i (t_i - y_i)^2$$

- For binary/categorical targets – we can use the **cross-entropy**, which measures the difference between two probability distributions for a given random variable.

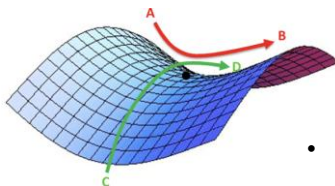
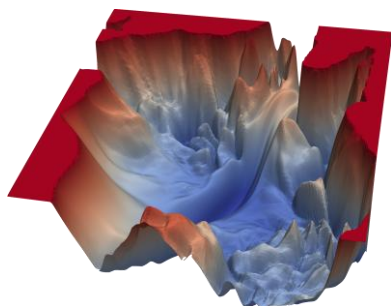
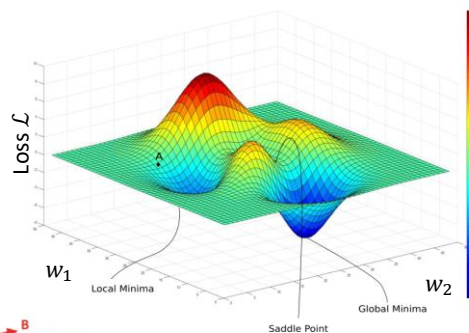
$$\mathcal{L} = H = - \sum_i t_i \log y_i$$



* Epoch with minimum validation $\mathcal{L}$

Loss surface

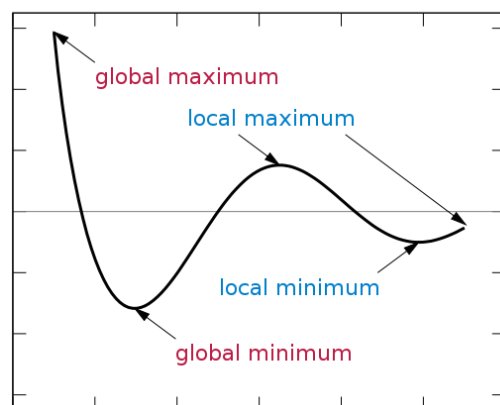
- A loss (or error) surface is produced a function of the “error” (e.g. RSS) vs weights.
- In general, it is not possible to plot one unless we had one neuron with 1 to 3 inputs.
- Consequently, we don’t know where the minima are in advance.



- Actual error surfaces are quite bumpy with many local minima and saddle points
- Gradient descent can’t find the global minima or tell whether it is or not.

Limitation of gradient descent with backpropagation

- Gradient descent with backpropagation is **not guaranteed** to find the global minimum of the error function, but only a **local minimum**.
- Also, it has **trouble crossing** plateaux in the error function landscape.
- This issue, caused by the non-convexity of error functions in neural networks, was long thought to be a **major drawback**.
- But in a 2015 review article^[*], the authors argue that in many practical problems, **it is not**.



[*] LeCun, et. al, “Deep learning” Nature, 521(7553):436-444, 2015.

Learning NNs

- Gradient descent:

- Batch gradient descent – update the weights using the full dataset, or part of the dataset (mini-batch).
- Stochastic gradient descent (SGD) – gradient descent over many small samples (instead of the full data). It is usually the **most powerful**.

$$\mathbf{v}(n) = \alpha \mathbf{v}(n-1) - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{w}(n)}$$

$$\Delta \mathbf{w}(n) = \mathbf{v}(n)$$

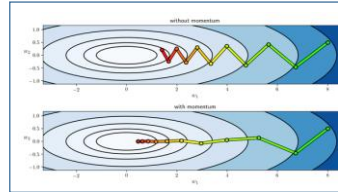
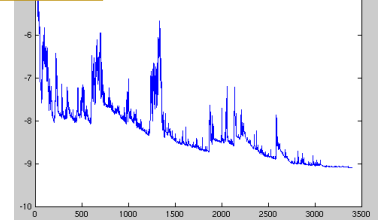
Where:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}(n)} = \delta_j^{(l)}(n) y_i^{(l-1)}(n)$$

$$\delta_j^{(l)}(n) = \begin{cases} e_j^{(l)}(n) \phi_j'(v_j^{(l)}(n)) & \text{if neuron } j \text{ in output layer } L \\ \phi_j'(v_j^{(l)}(n)) \sum_k \delta_k^{(l+1)}(n) w_{kj}^{(l+1)}(n) & \text{if neuron } j \text{ in hidden layer } l \end{cases}$$

* The momentum α is usually set to 0.9.

SGD error function fluctuates as weights are updated frequently.



Momentum works as viscosity, it **slows down** weight changes as learning algorithm approaches to a minimum.

Learning NNs

- RMSprop – Uses the expected decay of past gradients to decrease the learning rate over the algorithm iterations, seeking a similar effect as the SDG's momentum.

$$E \left[\left(\frac{\partial E}{\partial \mathbf{w}(n)} \right)^2 \right] = \alpha E \left[\left(\frac{\partial E}{\partial \mathbf{w}(n-1)} \right)^2 \right] + (1 - \alpha) \left(\frac{\partial E}{\partial \mathbf{w}(n)} \right)^2$$

Learning rate decreases as a function of the inverse square root of the expected decay (RMS)

$$RMS \left\{ E \left[\left(\frac{\partial E}{\partial \mathbf{w}(n)} \right)^2 \right] \right\} = \sqrt{E \left[\left(\frac{\partial E}{\partial \mathbf{w}(n)} \right)^2 \right] + \epsilon}$$

$$\Delta \mathbf{w}(n) = - \frac{\eta}{RMS \left\{ E \left[\left(\frac{\partial E}{\partial \mathbf{w}(n)} \right)^2 \right] \right\}} \frac{\partial E}{\partial \mathbf{w}(n)}$$

* Suggested default values: $\alpha = 0.9, \eta = 0.001$

Learning NNs

- Adaptive moment estimation (Adam) – an adaptive learning rate method (computes individual learning rates for different connections).

$$\mathbf{m}(n) = \beta_1 \mathbf{m}(n-1) + (1 - \beta_1) \frac{\partial \mathcal{L}}{\partial \mathbf{w}(n)} \quad \leftarrow 1^{\text{st}} \text{ momentum – decaying averages of past gradients}$$

$$\mathbf{v}(n) = \beta_2 \mathbf{v}(n-1) + (1 - \beta_2) \left(\frac{\partial \mathcal{L}}{\partial \mathbf{w}(n)} \right)^2 \quad \leftarrow 2^{\text{nd}} \text{ momentum (expected value) – decaying averages of past squared gradients}$$

$$\hat{\mathbf{m}}(n) = \frac{\mathbf{m}(n)}{1 - \beta_1} \quad \hat{\mathbf{v}}(n) = \frac{\mathbf{v}(n)}{1 - \beta_2} \quad \leftarrow \text{Corrections to avoid bias to zero}$$

$$\Delta \mathbf{w}(n) = - \frac{\eta}{\sqrt{\hat{\mathbf{v}}(n) + \epsilon}} \hat{\mathbf{m}}(n)$$

* Recommended values: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$

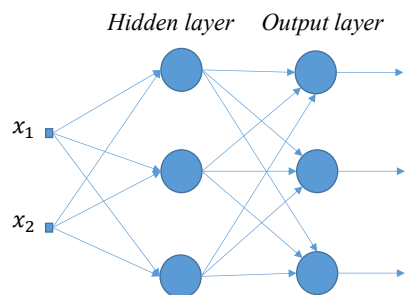
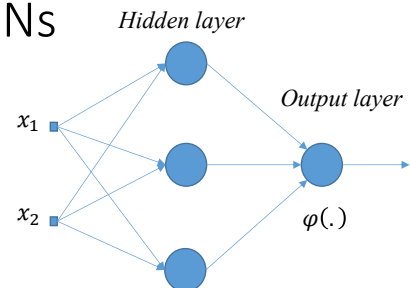
Dr Ivan Olier | AI

29

Regression and classification with NNs

- $\varphi(\cdot)$ is **linear** for regression tasks (e.g. $\varphi(v) = v$)
- $\varphi(\cdot)$ is either **sigmoid**, **tanh**, or equivalent, for a two-class classification task.
- A multi-output regression task is just an **extension** of the same architecture.
- It is also called “**multi-task learning**”.
- This network can also be used for multi-class classification tasks.
- The **softmax** activation function is typically used. It is seen as a **generalised** sigmoid, so the outputs are still probabilities:

$$y_i = \frac{e^{v_i}}{\sum_{i=1}^c e^{v_i}}$$



Dr Ivan Olier | AI

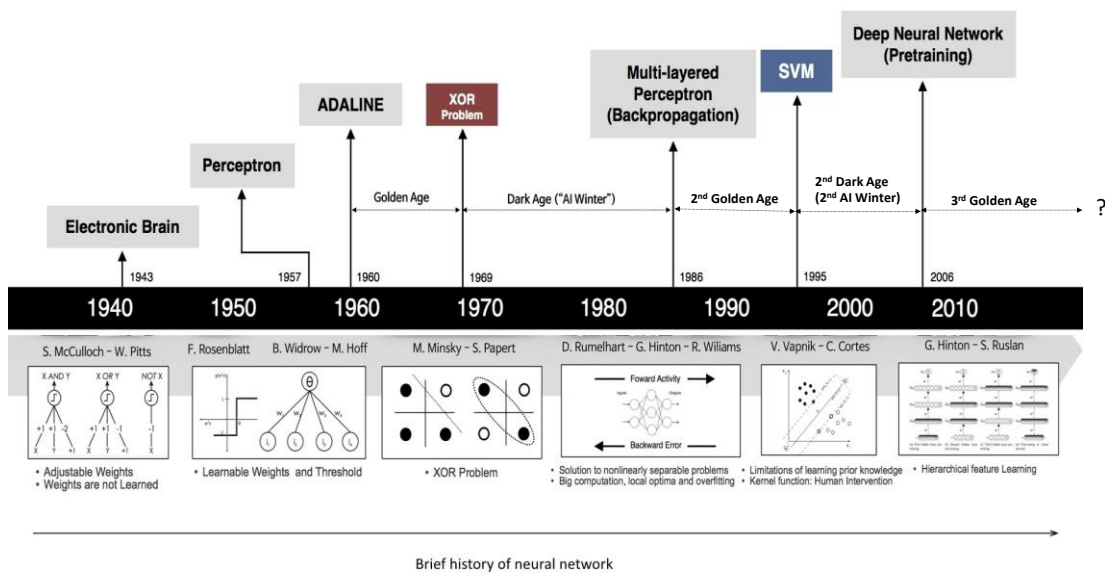
30

Deep Learning

Dr Ivan Olier | AI

31

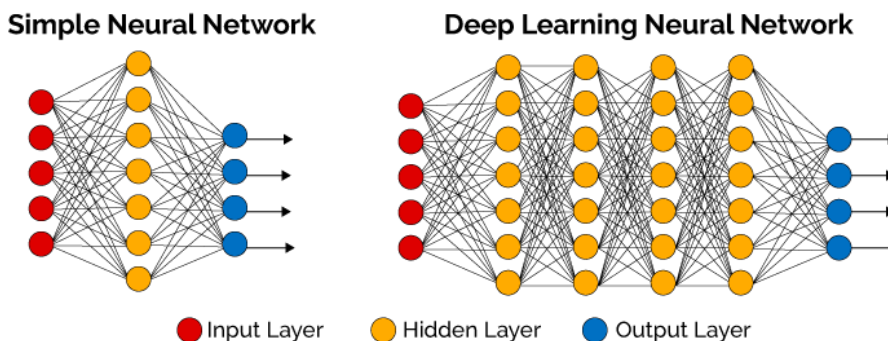
Brief history of neural network



Dr Ivan Olier | AI

32

From Neural Networks to Deep Learning



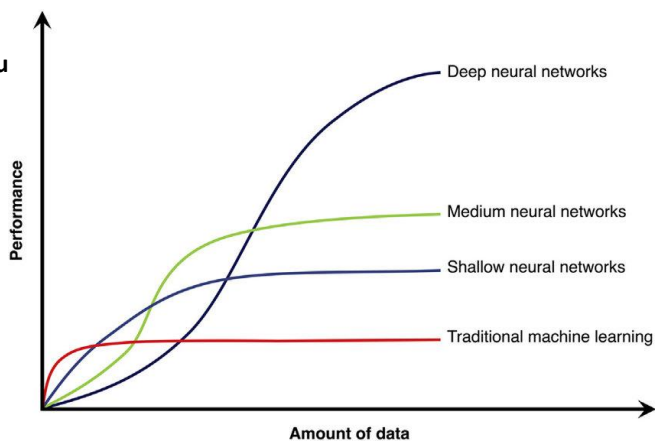
- Deep learning is actually a rebranding of artificial neural networks, but the term is usually reserved to architectures with several (many) layers.

Dr Ivan Olier | AI

33

Why is DL useful?

- Manually designed features are often **over-specified**, **incomplete** and take a **long time to design** and validate
- Learned Features are **easy to adapt**, **fast** to learn
- Deep learning provides a very **flexible**, (almost?) **universal** visual and linguistic information.
- Can learn both unsupervised and supervised
- Effective **end-to-end** joint system learning
- Utilise* large amounts of training data



* Although there are techniques that can help with it

Dr Ivan Olier | AI

34

Deep learning frameworks (just a few)



- Google's TensorFlow — arguably the most popular Deep Learning framework today. Gmail, Uber, Airbnb, Nvidia and lots of other prominent brands using it.
- Low-level API that requires lot of programming skills.



- It was developed for Facebook services but is already used for its own tasks by companies like Twitter and Salesforce.
- Likely the best option after TensorFlow, but not so robust.



- A highly scalable deep learning tool that can be used on a wide variety of devices.
- MXNet growth likely will be boosted by becoming an Apache project.



- High level API on top of TensorFlow.
- It is the best Deep Learning framework for those who are just starting out.

Dr Ivan Olier | AI

35

MNIST dataset

- The MNIST database (Modified National Institute of Standards and Technology database) is a large database of handwritten digits that is commonly used for training various image processing systems.
- It contains 60,000 training images and 10,000 testing images.
- In the original paper, LeCun et al. (1998)*, a support vector machine was used to get an **error rate of 0.8%**



* LeCun, Yann; Léon Bottou; Yoshua Bengio; Patrick Haffner (1998). "Gradient-Based Learning Applied to Document Recognition" (PDF). Proceedings of the IEEE.

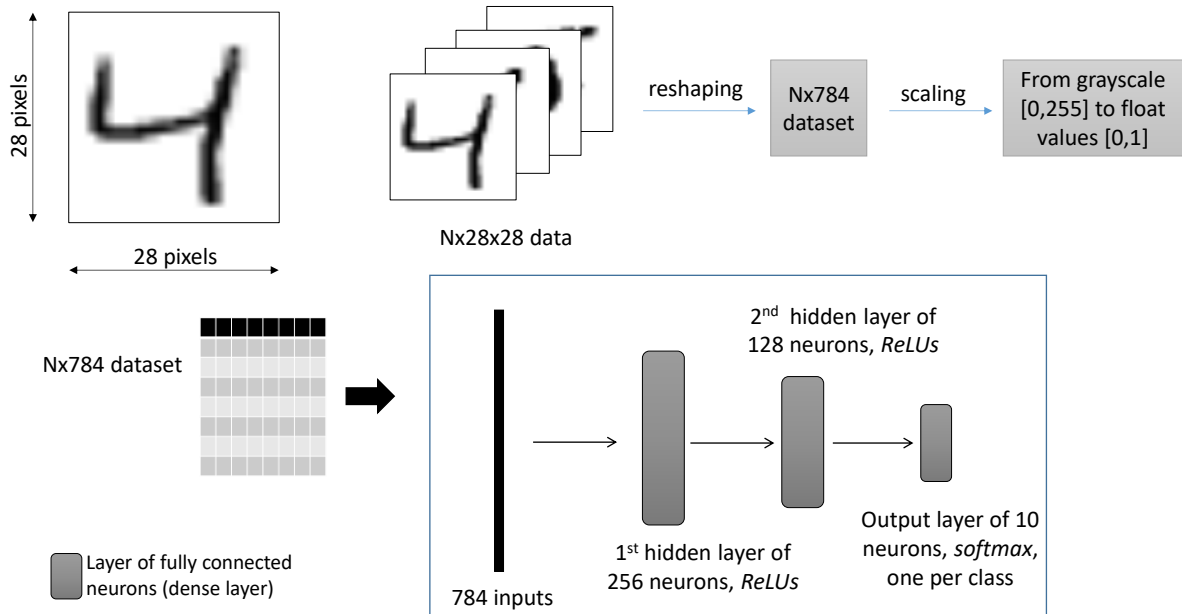
Dr Ivan Olier | AI

36

Type	Classifier	Error rate (%)
Linear classifier	Pairwise linear classifier	7.6 ^[9]
Non-linear classifier	40 PCA + quadratic classifier	3.3 ^[9]
Random Forest	Fast Unified Random Forests for Survival, Regression, and Classification (RF-SRC) ^[23]	2.8 ^[24]
Deep neural network (DNN)	2-layer 784-800-10	1.6 ^[26]
Boosted Stumps	Product of stumps on Haar features	0.87 ^[22]
Deep neural network	2-layer 784-800-10	0.7 ^[26]
Support-vector machine (SVM)	Virtual SVM , deg-9 poly, 2-pixel jittered	0.56 ^[25]
K-Nearest Neighbors	K-NN with non-linear deformation (P2DHMDM)	0.52 ^[21]
Deep neural network	6-layer 784-2500-2000-1500-1000-500-10	0.35 ^[27]
Convolutional neural network (CNN)	6-layer 784-40-80-500-1000-2000-10	0.31 ^[15]
Convolutional neural network	6-layer 784-50-100-500-1000-10-10	0.27 ^[28]
Convolutional neural network	Committee of 35 CNNs, 1-20-P-40-P-150-10	0.23 ^[8]
Convolutional neural network	Committee of 5 CNNs, 6-layer 784-50-100-500-1000-10-10	0.21 ^{[17][18]}
Random Multimodel Deep Learning (RMDL)	10 NN-10 RNN - 10 CNN	0.18 ^[19]
Convolutional neural network	Committee of 20 CNNs with Squeeze-and-Excitation Networks ^[29]	0.17 ^[20]

Some ML methods used with the MNIST database

Modelling the MNIST data with Tensorflow

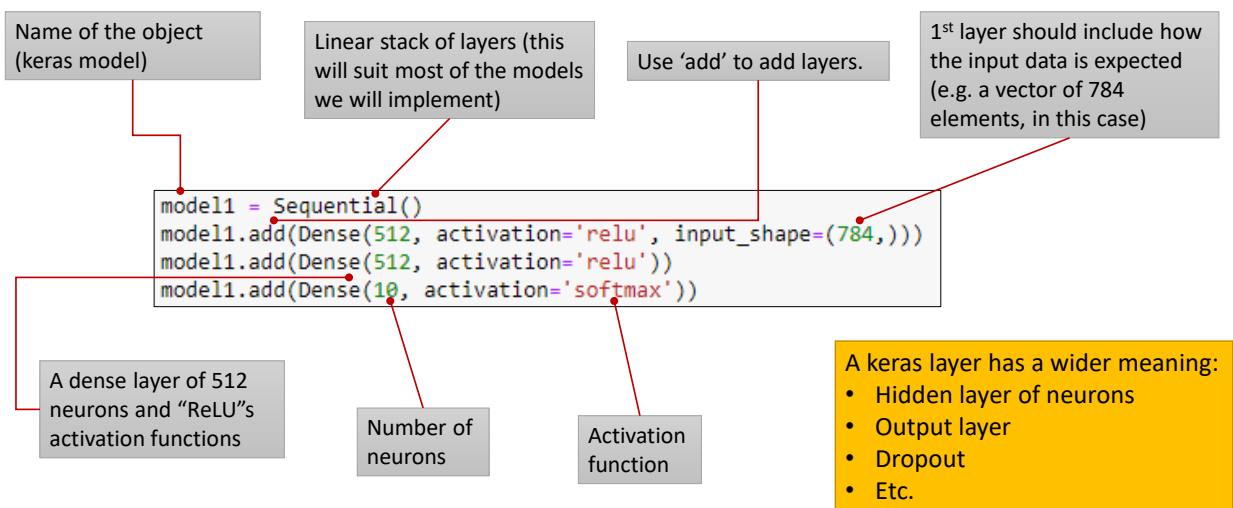


Dr Ivan Olier | AI

37

Modelling the MNIST data with Tensorflow

Step 1) To build the architecture



Dr Ivan Olier | AI

38

Modelling the MNIST data with Tensorflow

The keras object is easy to read:

Sequential model

```
model1.summary()
```

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 512)	401920
dense_1 (Dense)	(None, 512)	262656
dense_2 (Dense)	(None, 10)	5130
Total params: 669,706		
Trainable params: 669,706		
Non-trainable params: 0		

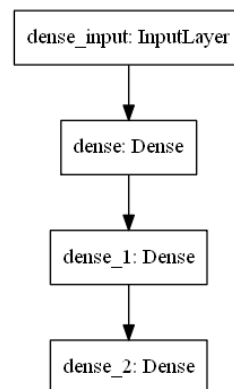
512 x 784 connections (weights) + 512 biases = 401,920

Layer name (you can change this too)

You might have pre-trained layers (more on this later)

Batch size (none in our case)

The architecture can be visualised:



Dr Ivan Olier | AI

39

Modelling the MNIST data with Tensorflow

Step 2) To set the optimiser

Loss function – the one that keras will use to assess convergence. More details in <https://keras.io/api/losses/>

```
model1.compile(
    loss='categorical_crossentropy',
    metrics=['accuracy'],
    optimizer='adam'
)
```

Metrics – not used to assess convergence

Optimisers :

- RMSprop - <https://keras.io/api/optimizers/rmsprop>
- SGD - <https://keras.io/api/optimizers/sgd>
- Adam - <https://keras.io/api/optimizers/adam>
- More? See Keras Reference : <https://keras.io/>

Dr Ivan Olier | AI

40

Modelling the MNIST data with Tensorflow

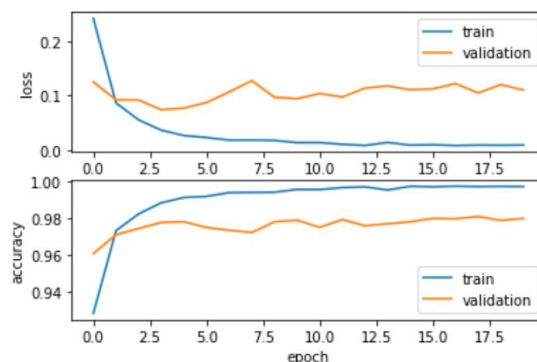
Step 3) To train the model

```
history = model1.fit(X_train, y_train,  
                    batch_size=128, epochs=20,  
                    verbose=1,  
                    validation_split=0.2)
```

You can let Keras set aside a validation set.

The train model is in "model1". You can optionally record training details in "history"

plot_history(history)



We may have overfitting !!!



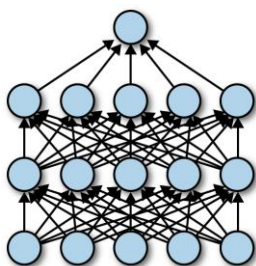
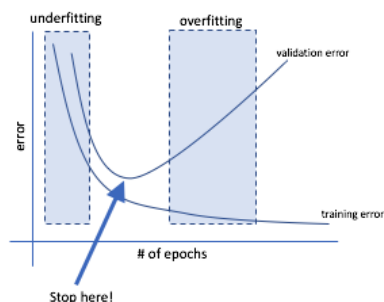
Dr Ivan Olier | AI

41

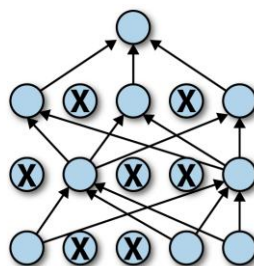
Ways to reduce overfitting

A large number of different methods have been developed:

- **Weight-decay** – where you try and keep the weights of the networks **small**. It will make the model **simpler**.
- **Early stopping** – where you try to stop the iterative training process when validation error starts increasing.
- **Dropout** – where you try and make your model more robust by randomly **dropping out** hidden units when you're training it.



(a) Standard Neural Net



(b) After applying dropout

Dr Ivan Olier | AI

42

Modelling the MNIST data with Tensorflow

Step 1B) To add dropout layers

```
from tensorflow.keras.layers import Dropout

model2 = Sequential()
model2.add(Dense(512, activation='relu', input_shape=(784,)))
model2.add(Dropout(rate=0.4))
model2.add(Dense(512, activation='relu'))
model2.add(Dropout(rate=0.4))
model2.add(Dense(10, activation='softmax'))

model2.summary()
```

Dropout layers – “rate” is the dropout out rate [0, 1]

Dr Ivan Olier | AI

43

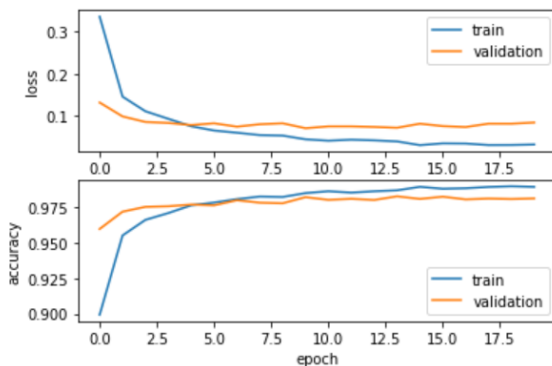
Modelling the MNIST data with Tensorflow

Model: "sequential_1"

Layer (type)	Output Shape	Param #
dense_3 (Dense)	(None, 512)	401920
dropout (Dropout)	(None, 512)	0
dense_4 (Dense)	(None, 512)	262656
dropout_1 (Dropout)	(None, 512)	0
dense_5 (Dense)	(None, 10)	5130
Total params: 669,706		
Trainable params: 669,706		
Non-trainable params: 0		

```
test_scores_model2 = model2.evaluate(X_test, y_test, verbose=0)
print("Test loss model 2:", test_scores_model2[0])
print("Test accuracy model 2:", test_scores_model2[1])
train_scores_model2 = model2.evaluate(X_train, y_train, verbose=0)
print("Train loss model 2:", train_scores_model2[0])
print("Train accuracy model 2:", train_scores_model2[1])
```

```
Test loss model 2: 0.0646570548415184
Test accuracy model 2: 0.98489997111816
Train loss model 2: 0.02108081988990307
Train accuracy model 2: 0.9950500130653381
```



- The overfitting level is **now reduced** in the new model
- We can also set aside a test set (for instance if we are performing **out-of-sample** validation)

<https://keras.rstudio.com/index.html>

Dr Ivan Olier | AI

44

Summary

- Modelling with Artificial Neural Networks (ANNs)
- Decide network architecture:
 - Regression: 1 (likely linear) output neuron
 - Classification: # output neurons = # classes (or # classes – 1), activation functions: sigmoids
 - Hidden layers: (theory) 1 hidden layer is sufficient. (practice) complex tasks could benefit from using more hidden layers. Number of neurons: several options should be tested. Same for the selection of the activation functions, but 'ReLU's should be sufficient.
- Decide optimiser: SGD, ADAM, others. Typically 'ADAM' is the choice.
- Decide other settings: number of epochs? Batch size? Learning rate? Momentum? How to reduce overfitting?
- Split your data: training, validation, test subsets
- Train your model using the training set, validate it using the validation set and test it using the test set.